Speech Denoising Using Time–Frequency Filtering

Project Topic Finalized – Revised Draft November 7, 2026

Members: Dayong Fang, Leyu Jiang, Jingyu Liu, Siqi Gong

1. Selected idea and response to feedback

We select **Idea 1: Speech Denoising**. Our central question is: *How do noise conditions and suppression strength affect the trade-off between background-noise reduction and speech preservation?* The motivation is noisy lecture recordings and online meetings. The main study uses classical DSP on single-channel recordings; a reference-based adaptive filtering experiment is a separate extension.

We reviewed the recommended Noise Cancellation project, which addressed chatter in instrumental recordings with multi-stage processing and an STFT-based U-Net [6]. Our project instead studies *speech preservation* with classical single-channel methods on condition-balanced clean/noisy speech pairs.

This progress report responds to all three instructor follow-ups: (1) finalized data and conditions, (2) additional filtering methods, and (3) actual spectrograms identifying mostly noise-dominant regions. Sections 2–3 document completed data inspection; Sections 4–5 describe methods and evaluation still to be implemented. No final denoising scores are claimed.

2. Dataset and fixed experimental scope

We use the University of Edinburgh paired clean/noisy speech database (**VoiceBank–DEMAND**) [1,2]. The official recordings are sampled at 48 kHz. We match clean/noisy files by basename and convert pairs to 16 kHz mono using identical antialiasing resampling. The downloaded test set contains 824 pairs from speakers p232 and p257. The group inspected the official condition logs and reported checking archive integrity.

Partition	Noise labels in the official logs	Nominal SNR (dB)
28-speaker training	babble, cafeteria, car, kitchen, meeting, metro, restaurant, station, traffic, ssn	0, 5, 10, 15
2-speaker test	bus, cafe, living, office, psquare	2.5, 7.5, 12.5, 17.5

The label `ssn` denotes speech-shaped noise; `babble` combines multiple speakers. The study describing the database identifies `cafe` as an open-area cafeteria. Train and test partitions use different speakers and noise conditions [2].

Tuning set: 400 pairs, chosen from the training condition log as 10 pairs for each of 10 noise labels at four SNR levels ($10 \times 4 \times 10$). **Held-out evaluation set:** 240 pairs from bus, office and cafe, with 20 pairs per noise/SNR cell ($3 \times 4 \times 20$), equally split between the two test speakers. MATLAB random seed 351 fixes both selection manifests. The three illustrated recordings below are excluded from evaluation selection. Each future method will use the same manifest and alignment rules.

Current status: Training and test audio and the condition logs have been obtained; the original report states the 400-pair tuning manifest has been finalized. Parameter tuning, enhancement, and held-out evaluation are pending. The revised analysis script adds explicit log-format checks before constructing manifests. Raw DEMAND noise remains an optional extension.

3. Completed waveform and spectrogram analysis

We inspected one real test recording from each selected noise category at nominal 2.5 dB SNR. These three examples are *exploratory*, not estimates of average performance. Let $s[n]$ be the clean reference, $y[n]$ the noisy recording and $v[n] = y[n] - s[n]$ the diagnostic residual. We did not independently peak-normalize the paired signals or fit a gain before subtraction. Pair shape, sample rate, and length are checked before resampling.

With a periodic Hann window $w[\ell]$ of $L = 400$ samples (25 ms), hop $H = 160$ samples (10 ms) and $K = 512$-point FFT,

$$X[k, m] = \sum_{\ell=0}^{399} x[mH + \ell]w[\ell]e^{-j2\pi k\ell/512}, \qquad k = 0, \ldots, 256. \tag{1}$$

The FFT-bin spacing is $f_s/K = 31.25$ Hz; zero-padding refines the display grid, not the intrinsic window resolution. All three spectrograms use 0–8 kHz and a common $[-90, -10]$ dB color range. Their values represent window-normalized STFT *magnitudes*, not power spectral densities. For a one-sided amplitude display, the positive-frequency interior bins are doubled, while the DC and Nyquist bins are not.

Identifying mostly noise-dominant regions. For each full analysis frame, we calculate

$$R_m = 10 \log_{10} \frac{\sum_{\ell=0}^{399} s[mH + \ell]^2 + \varepsilon}{\sum_{\ell=0}^{399} v[mH + \ell]^2 + \varepsilon}. \tag{2}$$

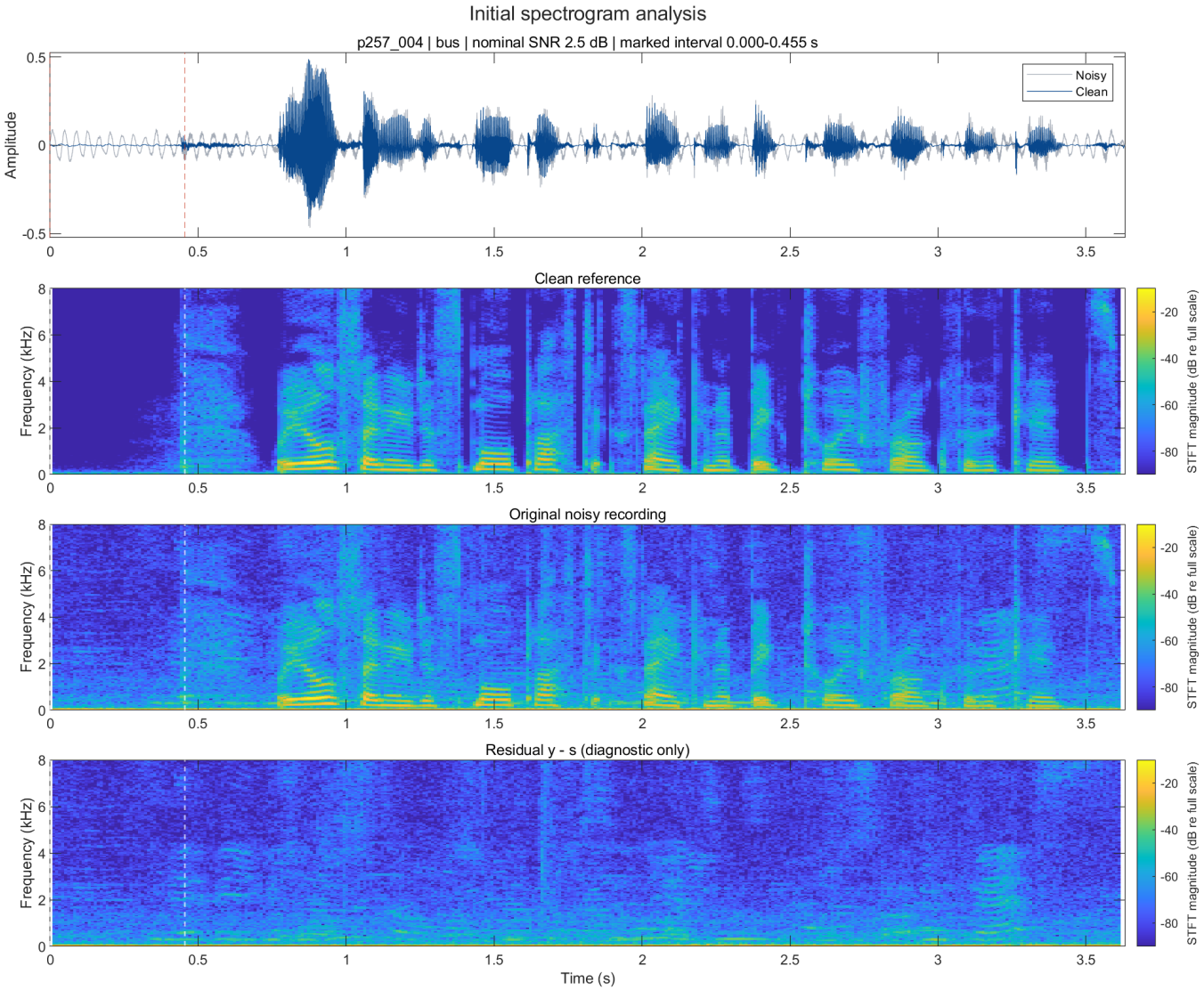
We mark the longest contiguous run satisfying $R_m \leq -10$ dB, including the full support of those frames. This means reference speech energy is no more than one tenth of diagnostic residual energy in each selected frame. The interval ratios below are recomputed from the underlying samples without double-counting overlapped frames. These intervals are *noise-dominant*, not necessarily silent.

Noise	Recording	Marked interval (s)	Ratio (dB)	Low-bin share*
bus	`p257_004`	0.000–0.455	−21.48	98.7%
office	`p257_017`	0.270–0.595	−17.21	99.3%
cafe	`p257_009`	0.000–0.595	−20.79	79.9%

*Original *unweighted nonnegative-frequency-bin* share below 300 Hz, $\sum_{k:f_k<300} \sum_m |V[k, m]|^2 / \sum_{k\geq 0} \sum_m |V[k, m]|^2$. These reported values are preserved from the original analysis, not silently replaced by recalculated results. They are descriptive concentration measures, *not* Parseval-corrected one-sided signal-power fractions or denoising scores. The revised MATLAB script also computes the corrected fraction, which should be reported only after rerunning the audio.

Interpretation. The paired plots distinguish target speech structure from background energy: noise is still present during low-speech intervals, and it overlaps frequencies used by speech. Thus fixed frequency removal alone cannot guarantee complete denoising without speech damage. The reference s and residual v are used *only* for these diagnostic plots and reference-based metrics. Neither will be used to detect pauses or update a real single-channel denoiser.

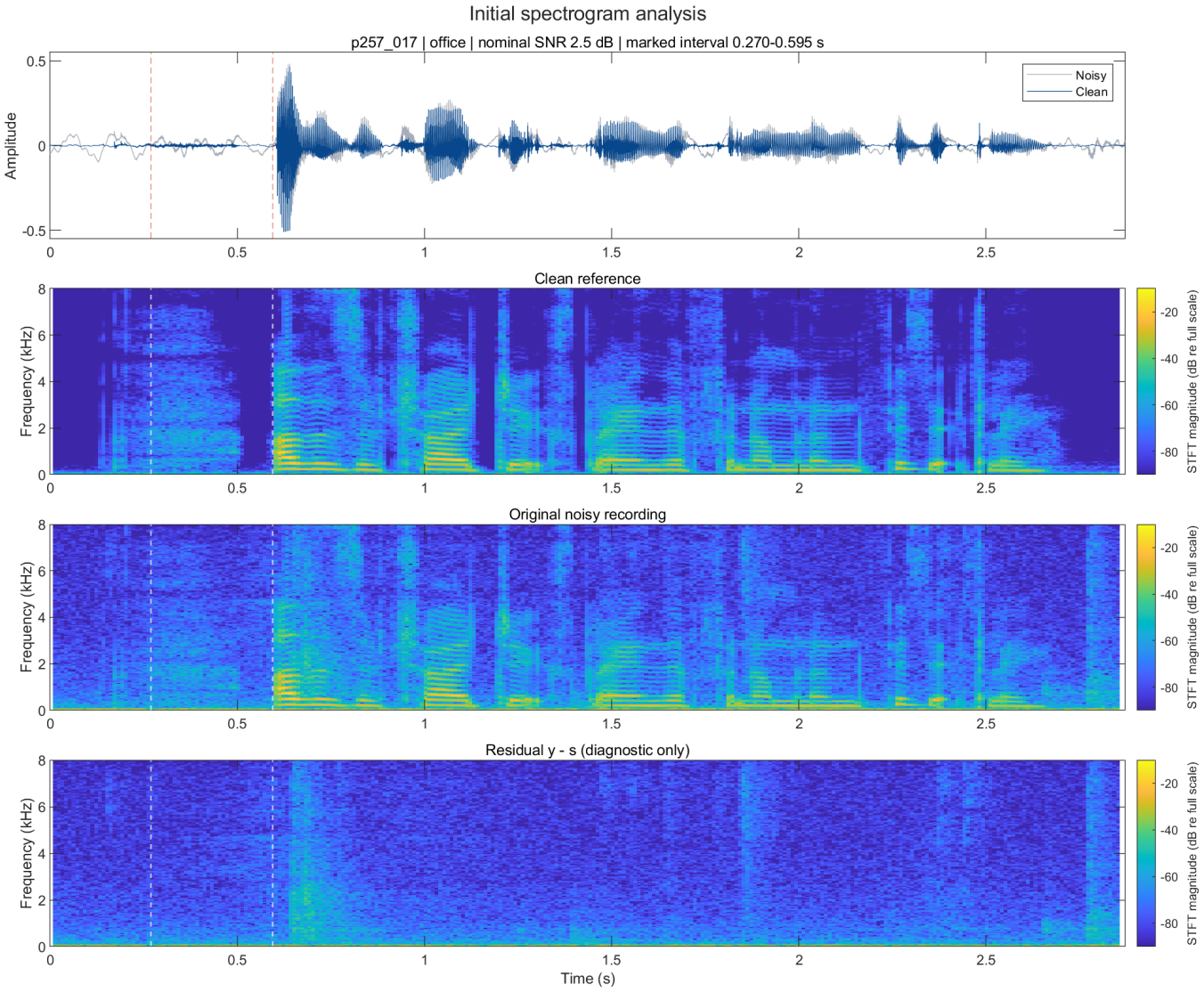
3.1 Bus example



Original MATLAB waveform and STFT analysis of `p257_004`. Vertical markers bound the reference-derived noise-dominant region.

Observation. Persistent low-frequency residual energy appears beneath and between speech structures. The marked 0.000–0.455 s interval has a speech-to-residual ratio of −21.48 dB. This motivates testing gentle low-frequency attenuation. However, energy above 300 Hz and low-frequency clean-speech content mean that a high-pass filter cannot be assumed to remove all noise without changing speech.

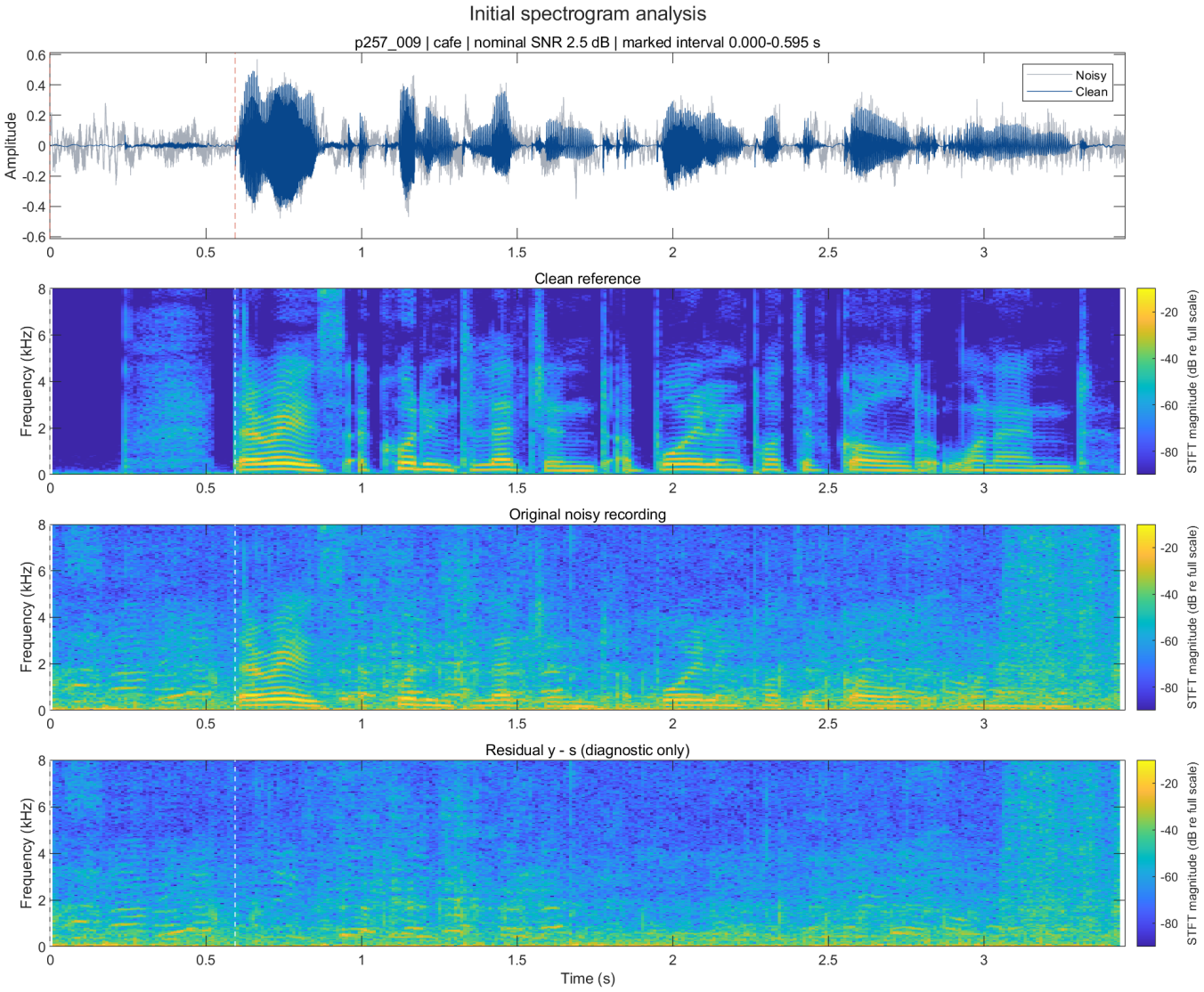
3.2 Office example



Original MATLAB waveform and STFT analysis of `p257_017`. The selected interval is diagnostic only.

Observation. The 0.270–0.595 s interval is dominated by residual energy (−17.21 dB). Its time–frequency patterns do not simply mirror every speech harmonic, offering a useful test of noise-estimation schemes. We will compare fixed and pause-updated estimates rather than treating the office label as proof that its spectrum is constant.

3.3 Cafe example



Original MATLAB waveform and STFT analysis of `p257_009`.

Observation. The marked 0.000–0.595 s region has relatively little reference speech (−20.79 dB compared with residual energy). Interference is visible over a wider part of the speech band, so a fixed cutoff cannot distinguish the talker from all background. Spectral gains may help, but weak speech and isolated musical-noise artifacts must be checked.

3.4 Cross-condition comparison (added)

The three examples suggest *different* denoising demands: the sampled bus and office residuals concentrate strongly in low-frequency bins, whereas the sampled cafe residual extends more broadly into the speech band. This provides a direct reason to compare a modest FIR high-pass baseline against time-varying spectral subtraction and Wiener gains. Filtering below a cutoff may reduce rumble but also remove voiced-speech energy; suppressing bins across the speech band may improve noise metrics while damaging quiet syllables. These are *hypotheses from three files*, not population-level claims about entire noise categories.

Study implication: Evaluate noise suppression *and* speech preservation on the same held-out files; do not rank methods using residual noise reduction alone.

4. Denoising methods to implement and compare

Unprocessed baseline. Keep $y[n]$ as the baseline for changes in SNR, STOI, and listening quality.

Conventional FIR filtering (additional method). Compare identity with 128th-order windowed FIR high-pass candidates at 100, 200, and 300 Hz, optionally adding a 7 kHz upper cutoff. These are *candidates*, not validated optimum cutoffs. Inspect each actual response with `freqz`, especially because order 128 may give a broad transition relative to a 100 Hz cutoff at 16 kHz sampling. Linear-phase candidates have 64 samples of group delay; align and crop all processed signals to a common valid interval before evaluation.

Power spectral subtraction. Assuming $Y = S + V$ with uncorrelated speech and noise,

$$\mathbb{E}|Y|^2 = P_s + P_v, \qquad \widehat{P}_s = \max\{|Y|^2 - \alpha\widehat{P}_v,\ \beta\widehat{P}_v\}, \quad \widehat{S} = \sqrt{\widehat{P}_s}\, e^{j\angle Y}. \tag{3}$$

We will compare subtraction strengths $\alpha \in \{1, 2, 4\}$ and floors $\beta \in \{0.01, 0.05\}$. Stronger subtraction can lower residual noise but may erase weak speech or create musical noise [3].

STFT-domain Wiener filtering. For a real gain G and uncorrelated sources,

$$J(G) = \mathbb{E}|S - GY|^2 = (1-G)^2 P_s + G^2 P_v, \qquad G_{\mathrm{W}} = \frac{P_s}{P_s + P_v} = \frac{\xi}{1+\xi}, \quad \xi = \frac{P_s}{P_v}. \tag{4}$$

Estimate the unknown powers from y alone, apply a small gain floor, and reconstruct using noisy phase and normalized overlap-add. We will contrast fixed noise estimates with updates during probable pauses [3].

Practical noise estimation (added). For each observed noisy frame, compute short-time energy

$$E_m = \frac{1}{L}\sum_{\ell=0}^{L-1} y[mH + \ell]^2. \tag{5}$$

As an initial *candidate* rule, select low-energy frames using an energy percentile threshold learned on the 400-pair tuning set (for instance, the lowest 20%, subject to validation). Estimate a starting $\widehat{P}_v[k]$ by averaging $|Y[k, m]|^2$ over these candidates. During later candidate pauses, update

$$\widehat{P}_v[k, m] = \lambda\widehat{P}_v[k, m-1] + (1-\lambda)|Y[k, m]|^2; \tag{6}$$

otherwise hold the previous estimate. The forgetting factor λ and threshold must be tuned. Low-energy speech may be misidentified as noise; recordings with no trustworthy pauses will be reported as a limitation. No clean-derived pause annotations from Section 3 will be fed to this detector.

LMS/NLMS adaptive cancellation (separate extension). The main corpus provides no independent noise-reference microphone. In a controlled simulation, instead generate $y = s + h * r$ with an independent noise-only reference r and known mixing path h. For reference vector $\mathbf{r}_n$,

$$e[n] = y[n] - \mathbf{w}_n^T\mathbf{r}_n, \qquad \mathbf{w}_{n+1} = \mathbf{w}_n + \mu e[n]\mathbf{r}_n \quad \text{(LMS)}, \qquad \mathbf{w}_{n+1} = \mathbf{w}_n + \frac{\mu e[n]\mathbf{r}_n}{\delta + \|\mathbf{r}_n\|^2} \quad \text{(NLMS)}. \tag{7}$$

The reference must correlate with added noise without containing target speech [4]. This extension has extra input information; its scores will be reported separately from the single-channel methods.

Expected trade-offs (to test, not measured yet): FIR may reduce rumble but color voiced speech; aggressive subtraction may remove noise and low-energy consonants; Wiener gains may offer smoother time–frequency suppression but remain sensitive to noise estimates. Parameters will be chosen by held-out-on-training validation, not by results on the final test set.

5. Evaluation, remaining work, and deliverable check

For clean speech s, noisy audio y and aligned output $\widehat{s}$, measured on the same valid samples,

$$\mathrm{SNR}_{\mathrm{in}} = 10\log_{10}\frac{\sum_n s[n]^2}{\sum_n (y[n]-s[n])^2}, \quad \mathrm{SNR}_{\mathrm{out}} = 10\log_{10}\frac{\sum_n s[n]^2}{\sum_n (\widehat{s}[n]-s[n])^2}, \quad \Delta\mathrm{SNR} = \mathrm{SNR}_{\mathrm{out}} - \mathrm{SNR}_{\mathrm{in}}. \tag{8}$$

The output error includes residual noise *and* speech distortion. No independent loudness normalization or optimal fitted gain will precede these metrics. Whole-utterance SNR includes pauses and therefore need not match the database's nominal active-speech SNR [2].

We will compute STOI relative to the clean reference and its change from the noisy baseline [5]. All methods will use the same files, alignment, STFT settings and valid comparison intervals. Report paired changes for all 12 test noise/SNR cells and, separately, for p232 and p257; two speakers do not establish broad generalization. Subjective listening at matched playback levels will check residual noise, muffling, consonant loss, and musical noise. STOI is an intelligibility proxy, not a complete perceptual-quality measure.

Parameter selection. Use the fixed 400-pair tuning manifest to choose parameters by mean STOI, with mean ΔSNR secondary and artifact checks on tuning examples. Freeze settings before using the 240-pair held-out evaluation manifest. Clean signals and residuals are only evaluation references; they must never define a denoiser's pause candidates or noise spectrum.

Instructor deliverable	**Evidence in this report**
Finalize dataset and conditions	Section 2: named data source, noise labels, SNRs, 400/240 selection procedure and excluded illustration files.
List additional methods	Section 4: FIR baseline, spectral subtraction, Wiener filtering, explicit noise-estimation proposal, and a separate reference-based LMS/NLMS extension.
Plot recordings and identify mostly noise	Section 3: three actual waveform/spectrogram comparisons, marked intervals, interval ratios and cross-condition interpretation.

Scope at submission. The three instructor follow-up tasks are addressed. Implementation/tuning of denoisers, processed-audio figures, improvement scores, listening studies and the group website remain subsequent milestones. A file report is permitted at this stage.

References

[1] C. Valentini-Botinhao, *Noisy speech database for training speech enhancement algorithms and TTS models*, University of Edinburgh, 2017. https://doi.org/10.7488/ds/2117.

[2] C. Valentini-Botinhao et al., "Speech Enhancement for a Noise-Robust Text-to-Speech Synthesis System using Deep Recurrent Neural Networks," *Interspeech*, 2016, Sec. 4.1. https://www.isca-archive.org/interspeech_2016/valentinibotinhao16_interspeech.pdf.

[3] T. Bäckström et al., *Introduction to Speech Processing*, "Noise attenuation." https://speechprocessingbook.aalto.fi/enhancement/noise-attenuation/.

[4] MathWorks, "Signal Enhancement Using LMS and NLMS Algorithms." https://www.mathworks.com/help/dsp/ug/enhance-a-signal-using-lms-and-normalized-lms-algorithms.html.

[5] MathWorks, "stoi: Short-time objective intelligibility measure." https://www.mathworks.com/help/audio/ref/stoi.html.

[6] Y. S. Hao, R. Kreucher, H. Lyu, and Y. Rong, "Noise Cancellation Final Project," EECS 351, Winter 2026. https://samhao0507.github.io/noise_cancelling_project_site/index.html.

Appendix: Revised MATLAB analysis script

The companion file `analyze_project_revised.m` includes the reproducible manifest selection, the three diagnostic plots, and explicit calculations of both the originally reported unweighted bin share and a corrected one-sided power fraction. It also prevents the DC and Nyquist magnitude bins from being doubled. The displayed figures on pages 3–5 are *preserved from the original report*; because raw audio was not supplied with this PDF revision, we have not claimed a rerun or substituted new empirical percentages. Minor endpoint color-bin differences may appear after rerunning this revised script. The official audio archives and logs must be placed in `data/`. MATLAB R2024a and Signal Processing Toolbox are used.

```
% EECS 351: initial VoiceBank-DEMAND analysis (no denoiser evaluated).
% MATLAB R2024a + Signal Processing Toolbox.
% Run this script from its folder with the extracted data/ directory.
clear; close all; clc;
root = fileparts(mfilename('fullpath'));
data = fullfile(root, 'data');
out = fullfile(root, 'figures');
if ~exist(out, 'dir'), mkdir(out); end
rng(351, 'twister');

% 1. Read official condition logs and validate their three-column layout.
train = readtable(fullfile(data, 'log_trainset_28spk.txt'), ...
    'FileType','text', 'Delimiter',' ', 'ReadVariableNames',false);
test = readtable(fullfile(data, 'log_testset.txt'), ...
    'FileType','text', 'Delimiter',' ', 'ReadVariableNames',false);
assert(width(train)==3 && width(test)==3, ...
    'Unexpected log format: inspect the official files/import options.');
train.Properties.VariableNames = {'File','Noise','NominalSNR'};
test.Properties.VariableNames = train.Properties.VariableNames;
train.File = string(train.File); train.Noise = string(train.Noise);
test.File = string(test.File); test.Noise = string(test.Noise);
assert(isnumeric(train.NominalSNR) && isnumeric(test.NominalSNR), ...
    'NominalSNR must be numerical; inspect the log import.');

% 2. Select reproducible, condition-balanced manifests.
examples = ["p257_004","p257_017","p257_009"];
conditions = ["bus","office","cafe"];
tuning = train([],:); evaluation = test([],:);
noises = unique(train.Noise);
for c = 1:numel(noises)
    for snr = [0 5 10 15]
        ix = find(train.Noise==noises(c) & train.NominalSNR==snr);
        assert(numel(ix)>=10, 'Insufficient tuning examples.');
        ix = ix(randperm(numel(ix),10));
        tuning = [tuning; train(ix,:)]; %#ok<AGROW>
    end
end
for c = 1:numel(conditions)
    for snr = [2.5 7.5 12.5 17.5]
        for speaker = ["p232","p257"]
            ix = find(test.Noise==conditions(c) & ...
                test.NominalSNR==snr & startsWith(test.File,speaker) & ...
                ~ismember(test.File,examples));
            assert(numel(ix)>=10, 'Insufficient evaluation examples.');
            ix = ix(randperm(numel(ix),10));
            evaluation = [evaluation; test(ix,:)]; %#ok<AGROW>
        end
    end
end
assert(height(tuning)==400 && height(evaluation)==240);
assert(~any(ismember(evaluation.File,examples)));
writetable(tuning, fullfile(root,'tuning_manifest.csv'));
writetable(evaluation, fullfile(root,'evaluation_manifest.csv'));

% 3. Extract comparable STFTs from three illustrative test recordings.
fs = 16000; L = 400; hop = 160; K = 512;
w = hann(L,'periodic');
summary = struct([]);
for c = 1:numel(examples)
    name = examples(c);
    [s,fs0] = audioread(fullfile(data,'clean_testset_wav',name+'.wav'));
    [y,fy] = audioread(fullfile(data,'noisy_testset_wav',name+'.wav'));
    assert(fs0==48000 && fy==fs0 && isequal(size(s),size(y)));
    assert(size(s,2)==1, 'Expected mono source files.');
    s = resample(s,1,3); y = resample(y,1,3);
    v = y-s;  % Diagnostic residual only; NEVER a denoiser input.
    starts = 1:hop:(numel(s)-L+1);
    idx = (0:L-1)' + starts;
    S = fft(s(idx).*w, K, 1);
    Y = fft(y(idx).*w, K, 1);
```

```
    V = fft(v(idx).*w, K, 1);
    S = S(1:K/2+1,:); Y = Y(1:K/2+1,:);
    V = V(1:K/2+1,:);
    tt = ((starts-1)+(L-1)/2)/fs;
    f = (0:K/2)'*fs/K;

    % Reference-derived annotation, for analysis only.
    es = mean(s(idx).^2,1); ev = mean(v(idx).^2,1);
    ratio = 10*log10((es+eps)./(ev+eps));
    mask = ratio <= -10;
    edges = diff([false mask false]);
    a = find(edges==1); b = find(edges==-1)-1;
    assert(~isempty(a), 'No noise-dominant interval found.');
    [~,j] = max(b-a+1);
    lo = (starts(a(j))-1)/fs;
    hi = (starts(b(j))-1+L)/fs;
    selection = floor(lo*fs)+1:min(numel(s),floor(hi*fs));
    localRatio = 10*log10(sum(s(selection).^2)/sum(v(selection).^2));

    % Preserve the ORIGINAL published diagnostic for traceability.
    pRaw = sum(abs(V).^2,2);
    oldBinShare = sum(pRaw(f<300))/sum(pRaw);
    % NEW: Parseval-consistent one-sided spectral power fraction.
    pPower = pRaw;
    pPower(2:end-1) = 2*pPower(2:end-1);
    correctedPowerFraction = sum(pPower(f<300))/sum(pPower);

    fig = figure('Visible','off','Color','w','Position',[50 50 1100 880]);
    layout = tiledlayout(4,1,'TileSpacing','compact','Padding','compact');
    nexttile;
    time = (0:numel(s)-1)'/fs;
    plot(time,y,'Color',[.65 .68 .72]); hold on;
    plot(time,s,'Color',[.03 .28 .55]);
    yl = max(abs([s;y]))*1.08;
    ylim([-yl yl]); xlim([0 time(end)]);
    xline(lo,'--','Color',[.8 .2 .1]);
    xline(hi,'--','Color',[.8 .2 .1]);
    ylabel('Amplitude'); legend('Noisy','Clean','Location','northeast');
    title(sprintf('%s | %s | nominal 2.5 dB | %.3f-%.3f s', ...
        name,conditions(c),lo,hi),'Interpreter','none');
    matrices = {S,Y,V};
    labels = {'Clean reference','Original noisy recording', ...
              'Residual y - s (diagnostic only)'};
    for k = 1:3
        nexttile;
        A = abs(matrices{k})/sum(w);
        A(2:end-1,:) = 2*A(2:end-1,:); % Do not double DC/Nyquist.
        db = 20*log10(max(A,1e-6));
        imagesc(tt,f/1000,db); axis xy;
        ylim([0 8]); xlim([0 time(end)]);
        clim([-90 -10]); colormap(fig,parula(256));
        cb = colorbar; cb.Label.String = 'STFT magnitude (dB re FS)';
        hold on; xline(lo,'w--'); xline(hi,'w--');
        ylabel('Frequency (kHz)'); title(labels{k});
    end
    xlabel('Time (s)'); title(layout,'Initial spectrogram analysis');
    exportgraphics(fig,fullfile(out,conditions(c)+'_analysis.png'), ...
        'Resolution',180);
    exportgraphics(fig,fullfile(out,conditions(c)+'_analysis.pdf'), ...
        'ContentType','vector');
    close(fig);

    summary(c).file = char(name);
    summary(c).noise = char(conditions(c));
    summary(c).duration_s = numel(s)/fs;
    summary(c).pause_start_s = lo;
    summary(c).pause_end_s = hi;
    summary(c).pause_speech_to_residual_dB = localRatio;
    summary(c).whole_utterance_SNR_dB = ...
        10*log10(sum(s.^2)/sum(v.^2));
    summary(c).original_unweighted_bin_share_lt300Hz = oldBinShare;
    summary(c).corrected_one_sided_power_fraction_lt300Hz = ...
        correctedPowerFraction;
end
fid = fopen(fullfile(root,'analysis_summary.json'),'w');
assert(fid>0, 'Could not write summary JSON.');
fprintf(fid,'%s',jsonencode(summary,PrettyPrint=true));
fclose(fid);
save(fullfile(root,'analysis_results.mat'), ...
    'summary','tuning','evaluation');
disp(struct2table(summary));
fprintf('Saved %d tuning and %d evaluation entries.\n', ...
    height(tuning),height(evaluation));
```